I. Dokumentacja do lab. nr 4 - "Enkapsulacja. Modyfikatory dostępu."

II. Roch Majkowski - grupa 1, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

1. Napisz klasę User z prywatnymi polami login, email, age oraz getterami i setterami z walidacją wieku i formatu e-mail.
2. Utwórz klasę Wallet z prywatnym polem balance i publicznymi metodami deposit(double) oraz pay(double) bez bezpośredniego dostępu do pola.
3. Zbuduj dwa pakiety: shop.core i shop.ui. Sprawdź, które elementy są widoczne przy użyciu public, protected i modyfikatora domyślnego.
4. Przygotuj klasę bazową Animal i klasę potomną Dog, a następnie pokaż praktyczny przypadek użycia protected.
5. Wybierz jedną istniejącą klasę z poprzednich laboratoriów i zrefaktoryzuj ją tak, by wszystkie pola były enkapsulowane.
6. Dopisz 2-4 zdania: jakie problemy może powodować pozostawienie pól jako public w większym projekcie.

V. Technologie wykorzystane w zadaniu

- Java

VI. Realizacja zadania

1. Zadanie - klasa `User`

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class User {
    private String login;
    private String email;
    private Integer age;
```

```
    public String getLogin() {
        return login;
    }

    public void setLogin(String login) {
        this.login = login;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        if (email != null && email.matches("^(?=.*@)(?=.*\\.).*$")) {
            this.email = email;
        } else {
            System.out.println("Email musi być poprawnie sformatowany!");
        }
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        if (age >= 0 && age <= 150) {
            this.age = age;
        } else {
            System.out.println("Wiek nie może być nierealny!");
        }
    }

    @Override
    public String toString() {
        return super.toString();
    }

}
```

Wywołanie kodu w klasie ▶ Main

```
public class Main {
    public static void main(String[] args) {

        // ========================================================= ZADANIE 1 ===================================


        User user_poprawny = new User(); // Inicjalizacja użytkownika z prawidłowymi danymi
        User user_niepoprawny = new User(); // Inicjalizacja użytkownika z NIEprawidłowymi danymi

        // Poprawne dane wejściowe
        user_poprawny.setLogin("JanKowalski123");
        user_poprawny.setAge(47);
        user_poprawny.setEmail("jankowalski123@gmail.com");
        System.out.println("Informacje o użytkowniku: \n" + "Login: " + user_poprawny.getLogin() + "\n" + "Wiek: '

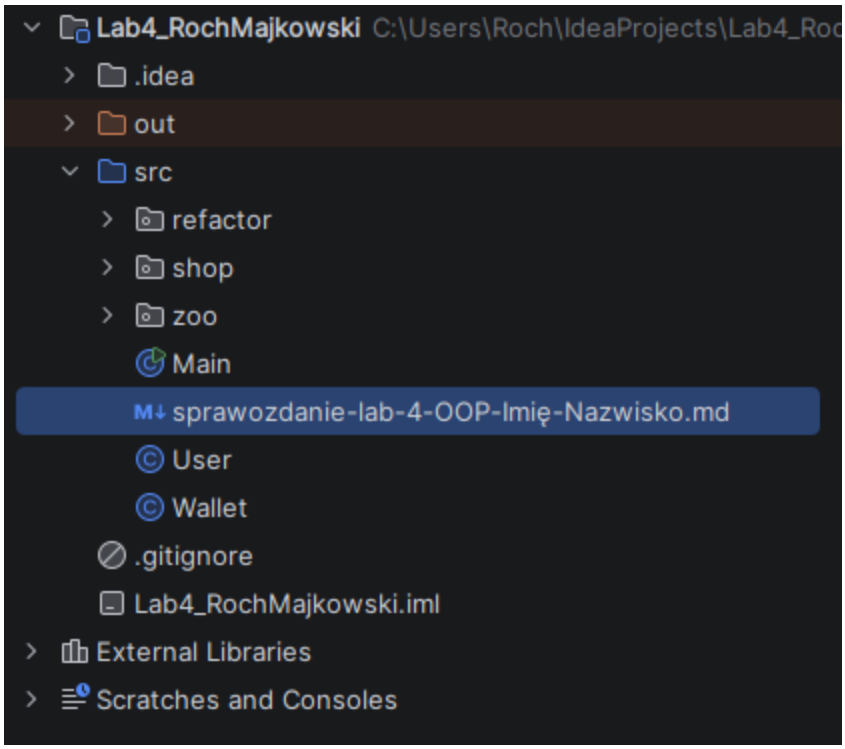
        // Niepoprawne dane wejściowe
        user_niepoprawny.setLogin("Dagmara");
```

```
        user_niepoprawny.setAge(-287);
        user_niepoprawny.setEmail("dagmarayahoo,com");
        System.out.println("\n");
    }
}
```

1a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



1b. Struktura projektu/programu:



2. Zadanie - klasa `Wallet`

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class Wallet {
    private double balance;

    public void deposit(double money) {
        if (money <= 0) {
            System.out.println("Kwota nie może być ujemna!");
        }
        balance += money;
    }

    public void pay(double money) {
        if (money <= 0) {
            System.out.println("Kwota nie może być ujemna!");
        }
        if (money > balance) {
            System.out.println("Nie posiadasz wystarczających środków");
        }
        balance -= money;
    }

    public Double getBalance() {
        return balance;
    }

}
```

Wywołanie kodu w klasie ▶ Main

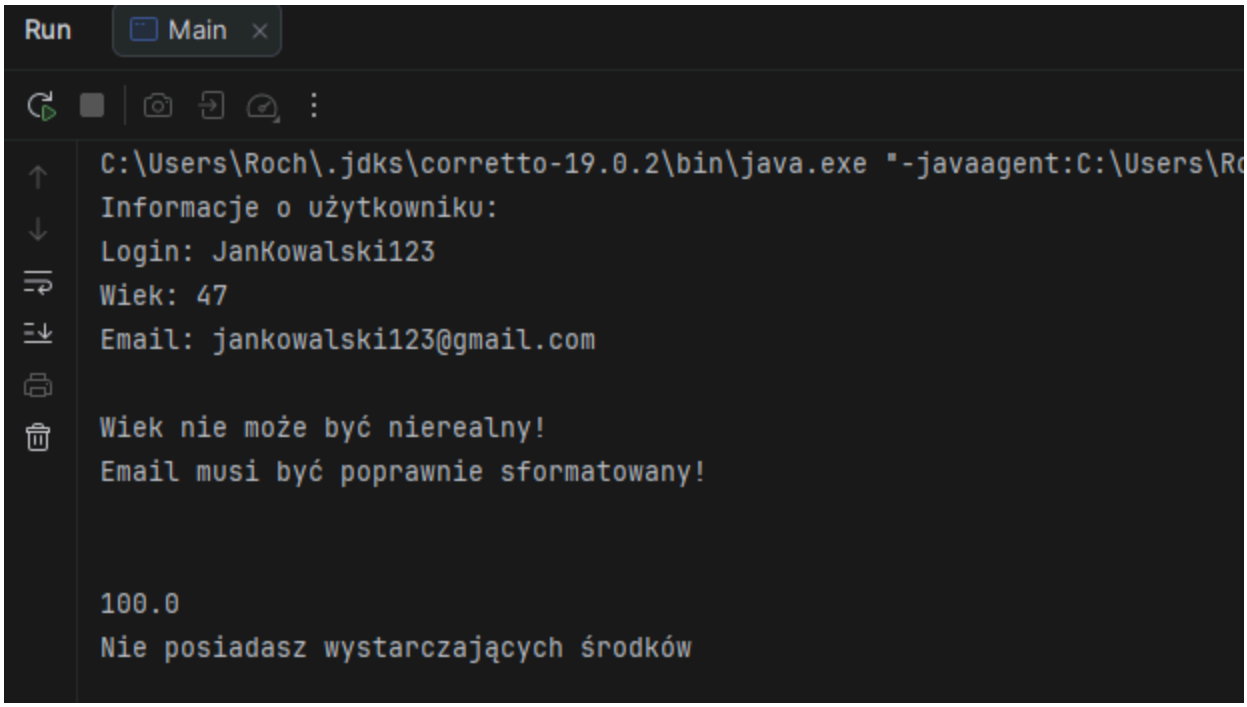
```
public class Main {
    public static void main(String[] args) {
        // ========================================================= ZADANIE 2 ================================


        Wallet wallet = new Wallet();
        wallet.deposit(250);
        wallet.pay(150);
        System.out.println(wallet.getBalance());

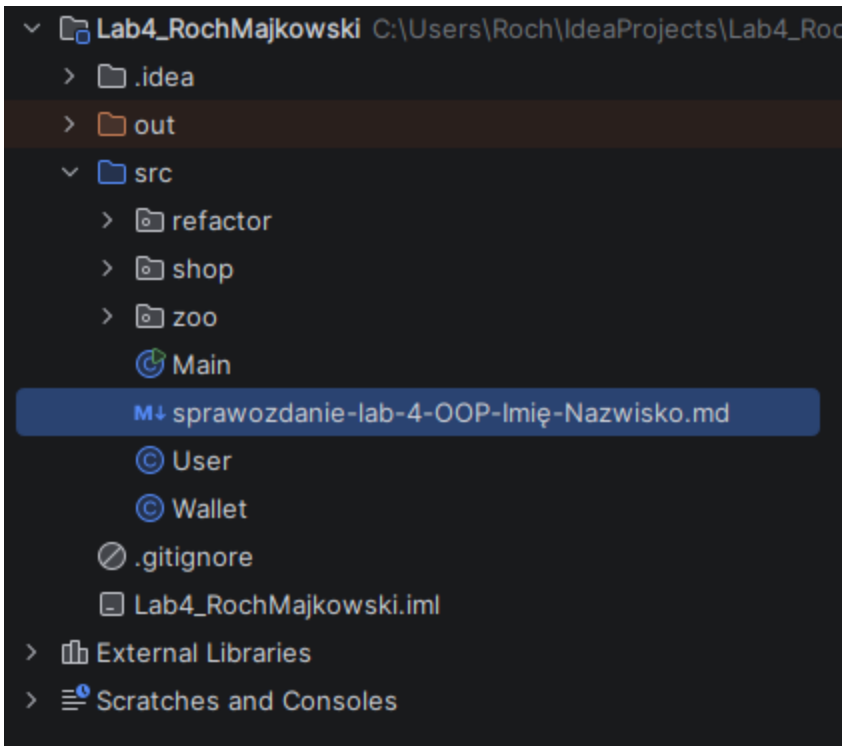
        // Nie ma wystarczająco środków
        wallet.pay(151);

        //wallet.balance = 1000; // ma prywatny dostęp nie można zmienić poza klasą danych
    }
}
```

2a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



2b. Struktura projektu/programu:



3. Zadanie - Pakiety `shop.core` i `shop.ui`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Product`

```
public class Product {
    public String publicField = "public";
    protected String protectedField = "protected";
    String defaultField = "default";
    private String privateField = "private";

    public void publicMethod() {}
    protected void protectedMethod() {}
    void defaultMethod() {}
    private void privateMethod() {}
}
```

Klasa ▶ Main

```
package shop.ui;
import shop.core.Product;

public class Main {
    public static void main(String[] args) {
        Product product = new Product();

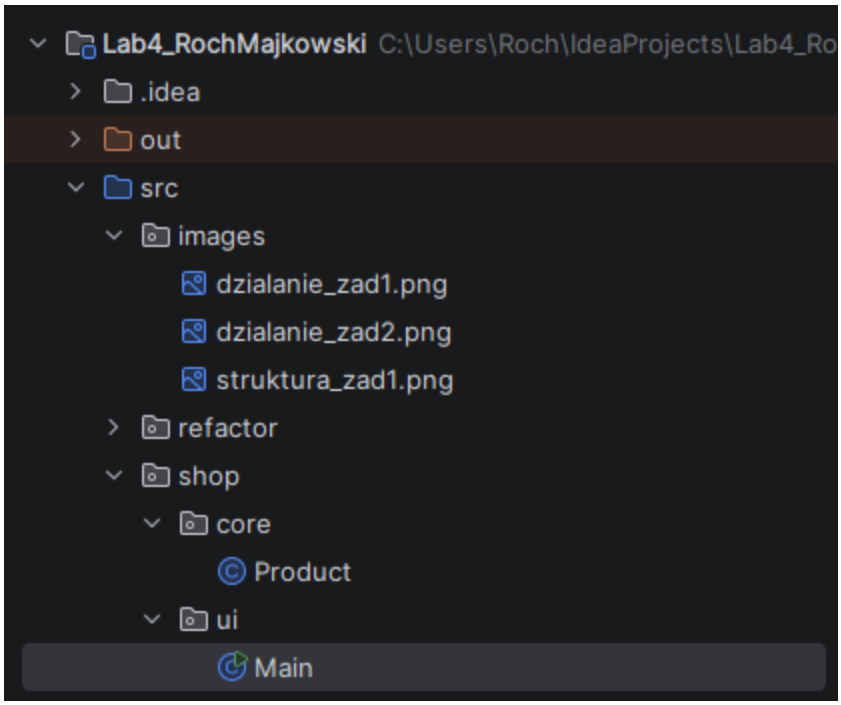
        //Public
        System.out.println(product.publicField); // Działa bez problemu
        product.publicMethod();

        //Protected
//        System.out.println(product.protectedField); // daje błąd : java: protectedField has protected access in
//        product.protectedMethod(); // daje błąd : java: protectedMethod() has protected access in shop.core.Proc

        //Default
//          System.out.println(product.defaultField); // daje błąd : java: defaultField is not public in shop.core
//          product.defaultMethod(); // daje błąd : java: defaultMethod() is not public in shop.core.Product; cann

        //Private
//        System.out.println(product.privateField); // daje błąd : java: privateField has private access in shop.c
//        product.privateMethod(); // daje błąd : java: privateMethod() has private access in shop.core.Product
    }
}
```

3a. Struktura projektu/programu:



3b. Wnioski:

- Z klasy w innym pakiecie, która nie dziedziczy po Product, dostępne są tylko elementy public,
- Elementy protected są widoczne dla podklas oraz w tym samym pakiecie,
- Elementy default tylko w tym samym pakiecie, a private tylko wewnątrz klasy Product.

4. `Animal` i `Dog` — modyfikator `protected`

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Animal`

```
package zoo.core;

public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }
}
```

Klasa `Dog`

```
package zoo.pets;

import zoo.core.Animal;
```

```java
public class Dog extends Animal {
    public Dog(String name){
        super(name);
    }

    public void sayMyName() {
        System.out.println(name + " szczeka: You're goddamn right.");
    }
}
```

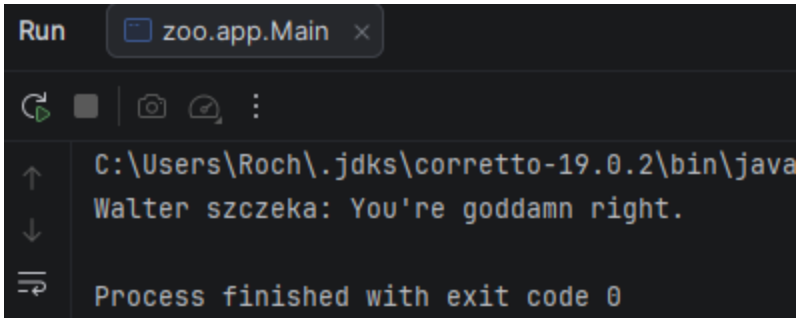
Klasa ▶ Main

```java
package zoo.app;
import zoo.pets.Dog;

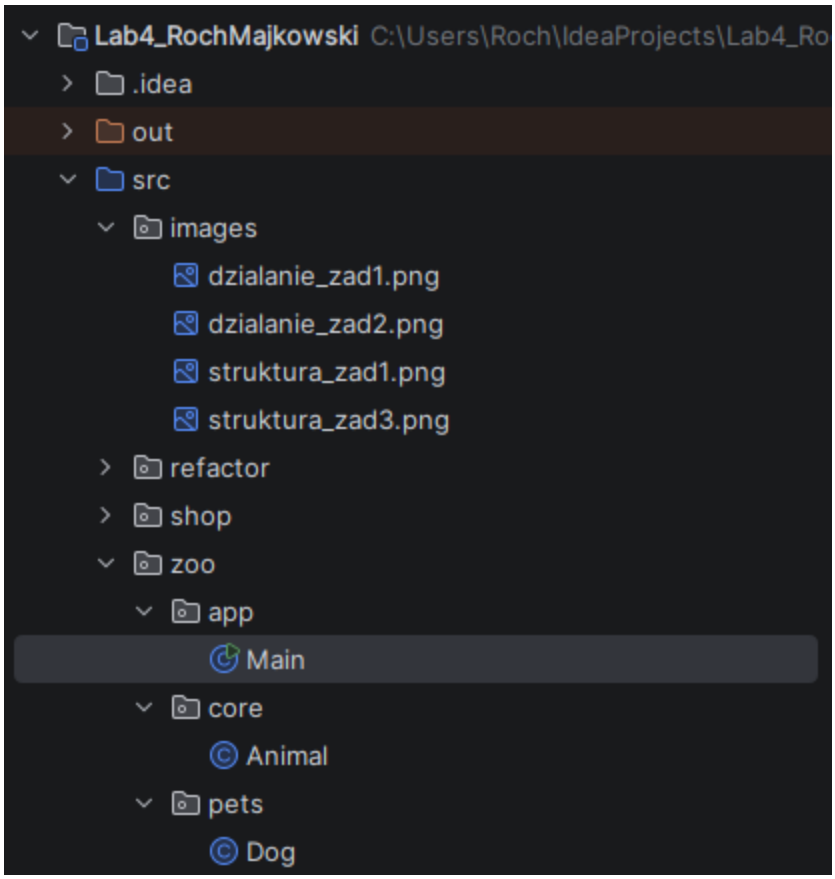
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("Walter");
        dog.sayMyName();

        //System.out.println(dog.name); // Błąd : java: name has protected access in zoo.core.Animal
    }
}
```

4a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



4b. Struktura projektu/programu:



5. Refaktoryzacja istniejącej klasy

Kod wykorzystany do rozwiązania zadania (zadań):

Klasa `Vehicle`

```
package refactor.core;

public class Vehicle {
    private String marka;
    private int rok_produkcji;

    public Vehicle(String marka, int rok_produkcji) {
        this.marka = marka;
        this.rok_produkcji = rok_produkcji;
    }

    public String getDescription() {
        return "Marka: " + marka + "\n" +
                "Rok produkcji: " + rok_produkcji;
    }
}
```

Klasa `Car`

```java
package refactor.car;
import refactor.core.Vehicle;

    public class Car extends Vehicle {
        private int liczba_drzwi;

        public Car(String marka, int rok_produkcji, int liczba_drzwi) {
            super(marka, rok_produkcji);
            this.liczba_drzwi = liczba_drzwi;
        }

        @Override
        public String getDescription() {
            return super.getDescription() + "\n" +
                    "Liczba drzwi: " + liczba_drzwi;   // element tylko dla klasy Car
        }
}
```

Klasa ▶ Main

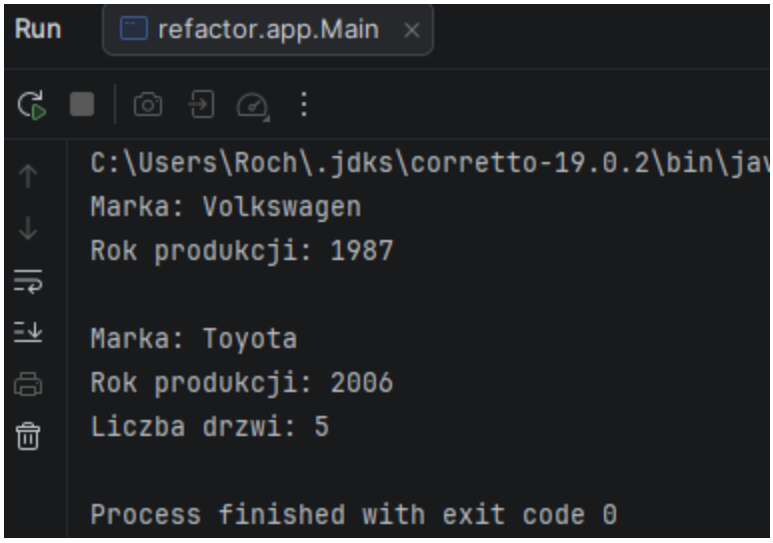
```java
package refactor.app;
import refactor.car.Car;
import refactor.core.Vehicle;

public class Main {
    public static void main(String[] args) {
        Vehicle vehicle = new Vehicle("Volkswagen", 1987);
        System.out.println(vehicle.getDescription());
        System.out.println();

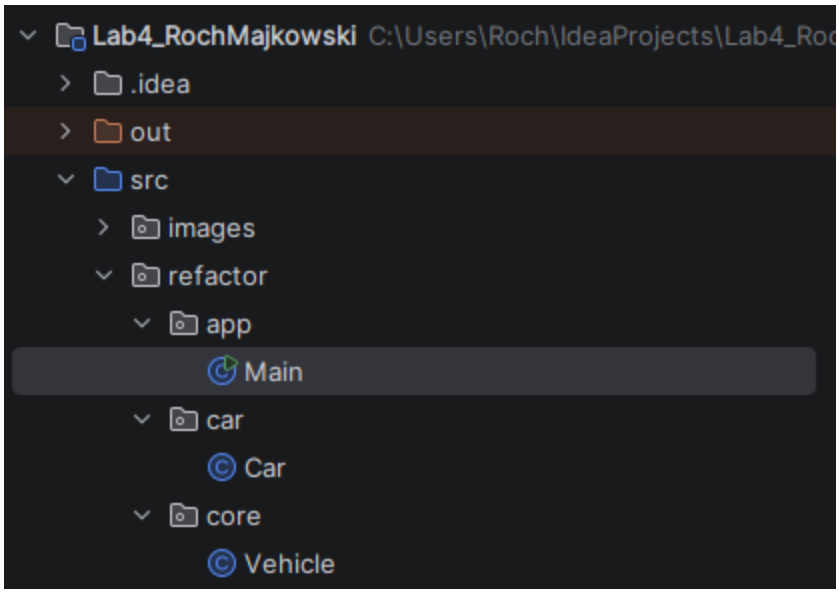
        Car car = new Car("Toyota", 2006, 5);
        System.out.println(car.getDescription());

        //vehicle.marka = "Fiat"; // Błąd: java: marka has private access in refactor.core.Vehicle
    }
}
```

5a. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



5b. Struktura projektu/programu:



5c. Wnioski:

Po refaktoryzacji:

- Wszystkie pola są private.
- Nie dodano żadnych getterów ani setterów. Wszystko przechodzi przez `getDescription()`, dopisując tylko liczbę drzwi.
- Klasy zostały oddzielone. ▶ `Main` trafił do osobnej klasy. Kod wewnątrz klasy ma dostęp do prywatnych pól. `vehicle.marka = "Fiat";` wywołuje błąd kompilacji.
- Konstruktory i metody są `public`

6. Problemy z polami `public`

6a. Wnioski

Pozostawienie pól jako `public` w większym projekcie łamię zasadę enkapsulacji, czyli ukrywa wewnętrzne informacje o obiekcie, udostępniając go przez metody. Brakuje walidacji; każdy fragment kodu może przypisać dowolną wartość np. `account.balance = 1000;`, nie ma jak tego odrzucić. W dużym kodzie może to doprowadzić do tego, że nie wiadomo, które fragmenty kodu modyfikują dane.